Collections and Generics in C#.Net



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



- Introduction to Collections
- Non-Generic Collection Classes
- Generic Collection Classes
- Generic Collection of User-Defined types

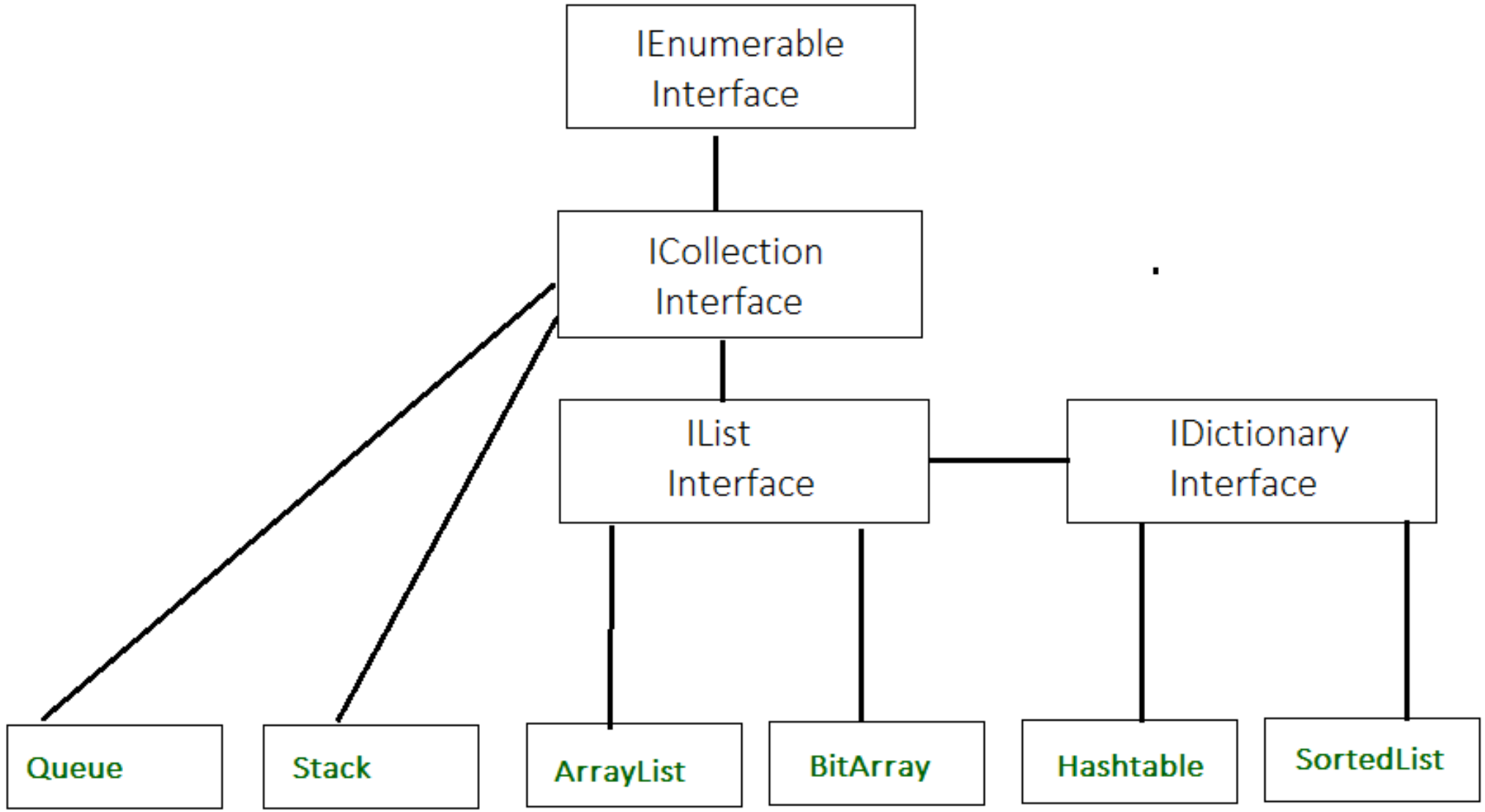
Collections

- In C#, a collection is a group of related objects that are grouped together and treated as a single unit.
- Collections provide a way to store, manipulate, and manage groups of data elements in a more structured and organized manner.
- They are part of the .NET Framework's class library and offer various data structures and algorithms for common programming tasks.
- The .NET Framework provides a wide range of collection classes in the **System.Collections (Non-Generic)** and **System.Collections.Generic (Generic)** namespaces.

Non-Generic Collections (System.Collections)

- C# also includes non-generic collection classes in the System.Collections namespace.
- These non-generic collection classes can be used to store and manipulate objects without specifying a specific type.
- Here are some commonly used non-generic collection classes in C#.
- The limitation of these Non-Generic collections is that while retrieving items, you need to cast into the appropriate data type, otherwise the program will throw a runtime exception.
- **It also affects on performance, because of boxing and unboxing.**

Non-Generic Collection Classes (System.Collections)	
ArrayList	Implements the IList interface using an array whose size is dynamically increased as required.
BitArray	Manages a compact array of bit values, which are represented as Booleans, where true indicates that the bit is on (1) and false indicates the bit is off (0).
Hashtable	Represents a collection of key/value pairs that are organized based on the hash code of the key.
Queue	Represents a first-in, first-out collection of objects.
SortedList	Represents a collection of key/value pairs that are sorted by the keys and are accessible by key and by index.
Stack	Represents a simple last-in-first-out (LIFO) non-generic collection of objects.


IEnumerable Interface
ICollection Interface
IList Interface
IDictionary Interface
Queue
Stack
ArrayList
BitArray
Hashtable
SortedList

- **ArrayList Class (System.Collections)**

•An ArrayList is a class. It uses an array whose size can be dynamically increased As required.

•ArrayList is more easier to work with multiple elements without bothering about index of an element.

•Array can only contain single type at time but ArrayList can contain variant type of data.

•Array size can not be re-declare but ArrayList can increase dynamically.

- https://learn.microsoft.com/en-us/dotnet/api/system.collections.arraylist?view=net-7.0

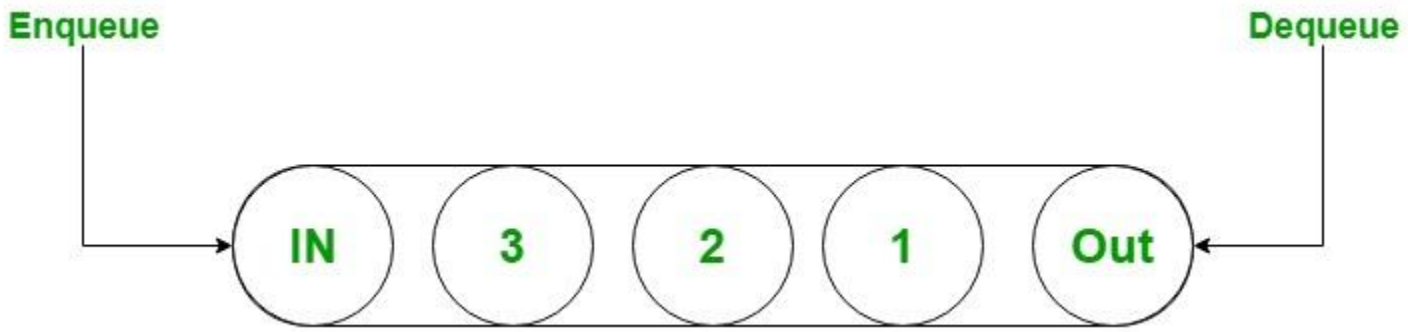
- **Example**

```
using System;
using System.Collections;
namespace Csharp8Features
{
    public class ArrayListDemo
    {
        public static void Main()
        {
          //Adding elements to ArrayList using Add() method
           ArrayList arrayList1 = new ArrayList();
           arrayList1.Add(101); //Adding Integer Value
           arrayList1.Add("James"); //Adding String Value
           arrayList1.Add("James"); //Adding Duplicate Value
           arrayList1.Add(" "); //Adding Empty
           arrayList1.Add(true); //Adding Boolean
           arrayList1.Add(4.5); //Adding double
           arrayList1.Add(null); //Adding null
           foreach (var item in arrayList1)
           {
              Console.WriteLine(item);
           }
```

```
 //Adding Elements to ArrayList using object
initializer syntax
           var arrayList2 = new ArrayList()
           {
              102, "Smith", "Smith", true, 15.6
           };
           foreach (var item in arrayList2)
           {
              Console.WriteLine(item);
           }
           Console.ReadKey();
       }
    }
}
```

• Queue(System.Collections)

- The Queue Class represents a FIFO(First In First Out) collection of Objects.
- The elements in a queue are added from one end and are removed from other.
- Queues are useful for storing messages in the order of which they arrive for sequential Processing.
- The first to be pushed is the first to be taken out.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.queue?view=net-7.0



- **Example**

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
        // Create a Queue to simulate a print job queue
        Queue printJobQueue = new Queue();

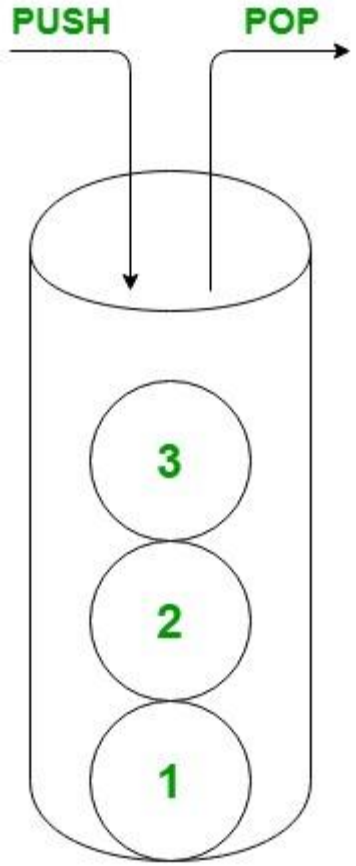
        // Enqueue print jobs
        printJobQueue.Enqueue("Document1.pdf");
        printJobQueue.Enqueue("Document2.docx");
        printJobQueue.Enqueue("Document3.txt");
```

```
// Simulate printing by dequeuing and processing print jobs
        while (printJobQueue.Count > 0)
        {
            string currentJob = (string)printJobQueue.Dequeue();
            Console.WriteLine("Printing: " + currentJob);
            // Simulate printing delay
            System.Threading.Thread.Sleep(2000);
        }

        Console.WriteLine("All print jobs completed.");
    }
}
```

- **Stack (System.Collections)**

- Elements are added and removed from the top of the stack.
- The last element added to the stack is the first one to be removed (last-in, first-out).
- It provides methods to push (add) elements onto the stack and pop (remove) elements from the stack.
- It supports peeking at the top element without removing it.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.stack?view=net-7.0



- **Example**

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
        // Create a Stack to simulate a browser history
        Stack browserHistory = new Stack();

        // Visit websites and add them to the history

browserHistory.Push("https://www.example.com");
        browserHistory.Push("https://www.google.com");
        browserHistory.Push("https://www.openai.com");

        // Simulate navigating back by popping the most
recent website from the history
        string currentWebsite =
(string)browserHistory.Pop();
        Console.WriteLine("Current Website: " +
currentWebsite);
```

```
// Simulate navigating back again
        currentWebsite = (string)browserHistory.Pop();
        Console.WriteLine("Current Website: " +
currentWebsite);

        // Simulate adding a new website to the history
        browserHistory.Push("https://www.github.com");

        // Peek at the most recent website in the history without
removing it
        string recentWebsite = (string)browserHistory.Peek();
        Console.WriteLine("Recent Website: " + recentWebsite);

        // Iterate over the browser history
        foreach (var website in browserHistory)
        {
            Console.WriteLine("Visited: " + website);
        }
    }
}
```

- **HashTable (System.Collections)**

A Hashtable is a data structure in C# that represents a collection of key-value pairs. It is part of the System.Collections namespace and provides efficient lookup, insertion, and deletion operations.

- In a Hashtable, each key is unique and is used to access its corresponding value.
- The keys and values can be of any data type.
- The Hashtable internally uses a hash function to generate a hash code for each key.
- This hash code is then used to determine the index or bucket where the key-value pair will be stored in the underlying data structure, typically an array.
- The primary advantage of using a Hashtable is its fast access to values based on their keys. Instead of iterating over the entire collection.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.hashtable?view=net-7.0

- **Example**

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
 // Create a Hashtable to store student grades
        Hashtable studentGrades = new Hashtable();

 // Add student names and corresponding grades to the
Hashtable
        studentGrades.Add("Alice", 95);
        studentGrades.Add("Bob", 87);
        studentGrades.Add("Charlie", 92);
        studentGrades.Add("Diana", 78);

 // Access and display the grade of a specific student
        int bobGrade = (int)studentGrades["Bob"];
        Console.WriteLine("Bob's Grade: " + bobGrade);

 // Modify the grade of a student
        studentGrades["Charlie"] = 88;
```

```
 // Check if a student is present in the Hashtable
        bool hasAlice =
studentGrades.ContainsKey("Alice");
        Console.WriteLine("Has Alice: " + hasAlice);

        // Remove a student and their grade from the
Hashtable
        studentGrades.Remove("Diana");

        // Iterate over the Hashtable
        foreach (DictionaryEntry entry in
studentGrades)
        {
            string studentName = (string)entry.Key;
            int grade = (int)entry.Value;
            Console.WriteLine(studentName + ": " +
grade);
        }
    }
}
```

- **SortedList (System.Collections)**

The SortedList<TKey, TValue>, and SortedList are collection classes that can store key-value pairs that are sorted by the keys based on the associated IComparer implementation. For example, if the keys are of primitive types, then sorted in ascending order of keys.

- Sorts elements as soon as they are added. Sorts primitive type keys in ascending order and object keys based on IComparer<T>.
- A key must be unique and cannot be null.
- A value can be null or duplicate.
- It is faster in the retrieval of data once sorted, whereas SortedDictionary<TKey, TValue> is faster in insertion and removing key-value pairs.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.sortedlist?view=net-7.0

- **Example**

```
using System;
using System.Collections;

class Program
{
    static void Main()
    {
        SortedList sortedList = new SortedList();

        // Adding key-value pairs to the sorted list
        sortedList.Add(3, "Apple");
        sortedList.Add(1, "Banana");
        sortedList.Add(2, "Orange");

        // Accessing elements in the sorted list
        Console.WriteLine(sortedList[1]);  // Output:
Banana
```

```
 // Iterating over the sorted list
        foreach (DictionaryEntry entry in sortedList)
        {
            Console.WriteLine($"{entry.Key}:
{entry.Value}");
        }

        // Removing elements from the sorted list
        sortedList.Remove(2);

        // Checking if a key exists in the sorted list
        bool containsKey = sortedList.ContainsKey(2);  //
false

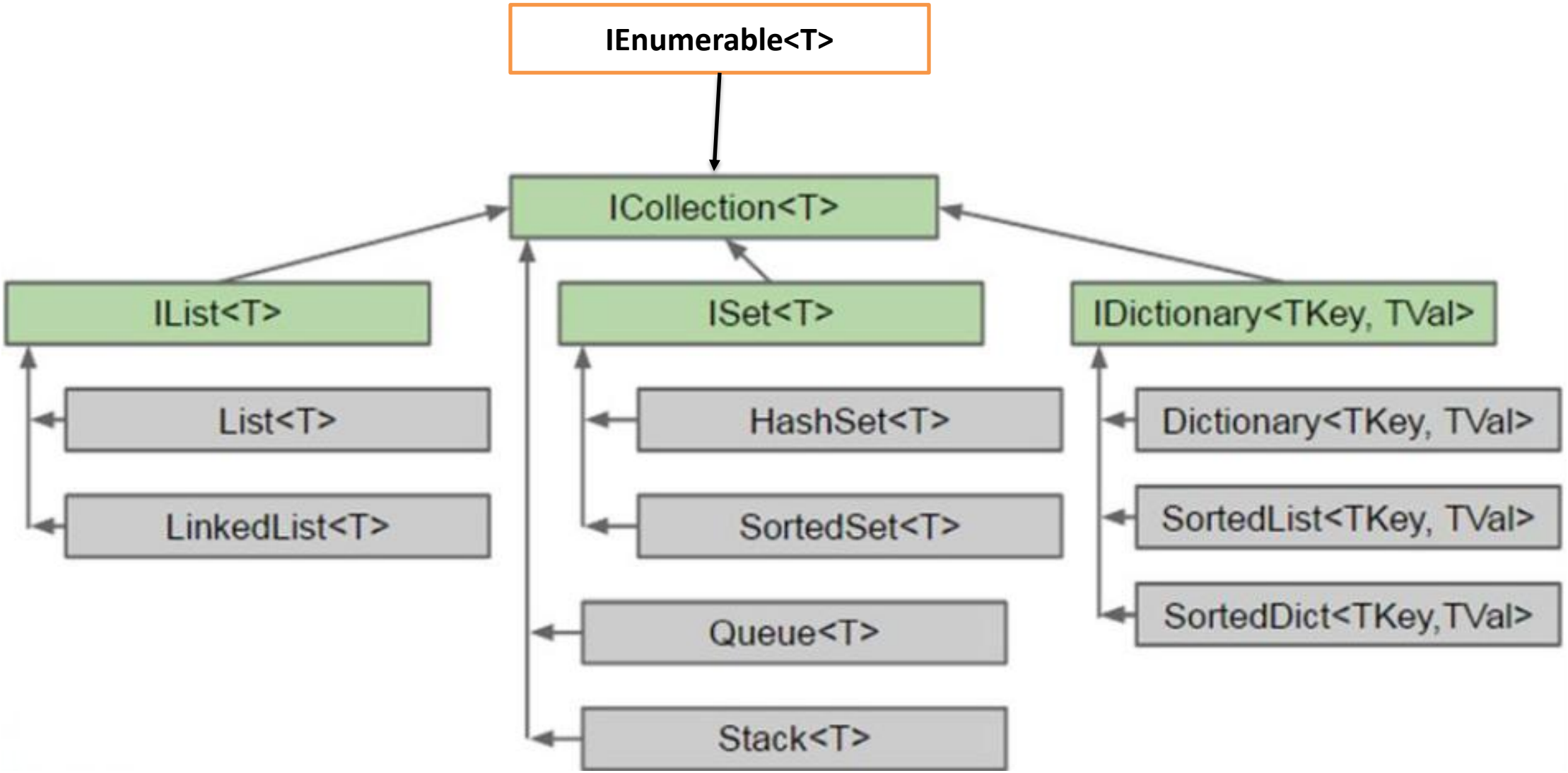
        // Checking if a value exists in the sorted list
        bool containsValue =
sortedList.ContainsValue("Apple");  // true
    }
}
```

Generic Collections (System.Collections.Generics)

- In C#, a generic collection is a type-safe collection that allows you to store and manipulate objects of a specific type. It is part of the .NET Framework's **System.Collections.Generic** namespace.
- Generic collections provide several advantages over non-generic collections, such as type safety and performance improvements.
- With generic collections, you can define the type of elements the collection will hold at compile-time.
- This ensures that only objects of the specified type or its derived types can be added to the collection, eliminating the need for explicit type casting and reducing the possibility of runtime errors.

The following are widely used generic collections:

Generic Collections Classes (System.Collections.Generic)	
List<T>	Generic List<T> contains elements of specified type. It grows automatically as you add elements in it.
Dictionary<TKey,TValue>	Dictionary<TKey,TValue> contains key-value pairs.
SortedList<TKey,TValue>	SortedList stores key and value pairs. It automatically adds the elements in ascending order of key by default.
Hashset<T>	Hashset<T> contains non-duplicate elements. It eliminates duplicate elements.
Queue<T>	Queue<T> stores the values in FIFO style (First In First Out). It keeps the order in which the values were added. It provides an Enqueue() method to add values and a Dequeue() method to retrieve values from the collection.
Stack<T>	Stack<T> stores the values as LIFO (Last In First Out). It provides a Push() method to add a value and Pop() & Peek() methods to retrieve values.


IEnumerable<T>
ICollection<T>
IList<T>
ISet<T>
IDictionary<TKey, TVal>
List<T>
LinkedList<T>
HashSet<T>
SortedSet<T>
Queue<T>
Stack<T>
Dictionary<TKey, TVal>
SortedList<TKey, TVal>
SortedDict<TKey,TVal>

- **List<T> (System.Collections.Generics)**

You have already learned about ArrayList in the previous section. An ArrayList resizes automatically as it grows.

- The List<T> collection is the same as an ArrayList except that List<T> is a generic collection whereas ArrayList is a non-generic collection.
- List<T> can contain elements of the specified type. It provides compile-time type checking and doesn't perform boxing-unboxing because it is generic.
- Elements can be accessed by passing an index e.g. myList[0]. Indexes start from zero.
- List<T> performs faster and less error-prone than the ArrayList.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=net-7.0

- **Example**

```
using System;
using System.Collections.Generic;

class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
}
class Program
{
    static void Main()
    {
        List<Person> people = new List<Person>();

        // Adding people to the list
        people.Add(new Person("Alice", 25));
        people.Add(new Person("Bob", 30));
        people.Add(new Person("Charlie", 35));
```

```
// Displaying the people
        Console.WriteLine("People:");
        foreach (Person person in people)
        {
            Console.WriteLine(person);
        }
        // Modifying a person's age
        people[1].Age = 32;
        // Checking if a person exists in the list
        bool contains = people.Contains(new Person("Alice",
25));  // true

        // Sorting the people by age
        people.Sort((p1, p2) =>
            p1.Age.CompareTo(p2.Age));

        // Displaying the sorted people
        Console.WriteLine("\nSorted People (by Age):");
        foreach (Person person in people)
        {
            Console.WriteLine(person);
        }
    }
}
```

• Dictionary<TKey, TValue> (System.Collections.Generics)

The Dictionary<TKey, TValue> collection in C# is same as English dictionary.

- English dictionary is a collection of words and their definitions, often listed alphabetically in one or more specific languages.In the same way, the Dictionary in C# is a collection of Keys and Values, where key is like word and value is like definition.
- Dictionary<TKey, TValue> stores key-value pairs.
- Keys must be unique and cannot be null.
- Values can be null or duplicate.
- Values can be accessed by passing associated key in the indexer e.g. myDictionary[key].
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2?view=net-7.0

- **Example**

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        Dictionary<string, int> studentGrades = new
Dictionary<string, int>();

        // Adding student names and grades to the
dictionary
        studentGrades.Add("Alice", 90);
        studentGrades.Add("Bob", 85);
        studentGrades.Add("Charlie", 92);

        // Accessing grades in the dictionary
        Console.WriteLine("Bob's Grade: " +
studentGrades["Bob"]);  // Output: Bob's Grade:
85
```

```
// Modifying grades in the dictionary
        studentGrades["Alice"] = 95;

        // Checking if a student exists in the dictionary
        bool containsStudent =
studentGrades.ContainsKey("Charlie");  // true

        // Removing students from the dictionary
        studentGrades.Remove("Bob");

        // Iterating over the dictionary
        foreach (KeyValuePair<string, int> entry in
studentGrades)
        {
            Console.WriteLine("Name: " + entry.Key + ",
Grade: " + entry.Value);
        }
    }
}
```

- **SortedList<TKey, TValue> (System.Collections.Generics)**

The generic SortedList SortedList<TKey, TValue> represents a collection of key-value pairs that are sorted by key based on associated IComparer<T>.

- A SortedList collection stores key and value pairs in ascending order of key by default.
- A key must be unique and cannot be null.
- A value can be null or duplicate.
- A value can be accessed by passing associated key in the indexer mySortedList[key]
- It uses less memory than SortedDictionary<TKey,TValue>.
- **It is faster in the retrieval of data once sorted, whereas SortedDictionary<TKey, TValue> is faster in insertion and removing key-value pairs**.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.sortedlist-2?view=net-7.0

- **Example**

```
using System;
using System.Collections.Generic;

class Task
{
    public string Name { get; set; }
    public DateTime Deadline { get; set; }

    public Task(string name, DateTime deadline)
    {
        Name = name;
        Deadline = deadline;
    }

    public override string ToString()
    {
        return $"{Name}: {Deadline.ToShortDateString()}";
    }
}
```

- **Example**

```
class Program
{
   static void Main()
   {
      SortedList<DateTime, Task> tasksByDeadline = new SortedList<DateTime, Task>();
      // Adding tasks to the sorted list
      tasksByDeadline.Add(DateTime.Parse("2023-06-25"), new Task("Complete Report",
DateTime.Parse("2023-06-25")));
      tasksByDeadline.Add(DateTime.Parse("2023-06-20"), new Task("Submit Proposal",
DateTime.Parse("2023-06-20")));

      // Accessing tasks in the sorted list
      Console.WriteLine("Task due on June 20th: " + tasksByDeadline[DateTime.Parse("2023-06-20")]);
     // Output: Submit Proposal: 6/20/2023

      // Removing tasks from the sorted list
      tasksByDeadline.Remove(DateTime.Parse("2023-06-20"));

      // Iterating over the sorted list
      foreach (KeyValuePair<DateTime, Task> kvp in tasksByDeadline)
      {
         Console.WriteLine(kvp.Value);
      }}}
```

- **Queue<T> (System.Collections.Generics)**

In C#, the Queue<T> class is a generic collection class that represents a first-in, first-out (FIFO) collection of objects.

- It provides methods for adding elements to the end of the queue and removing elements from the beginning of the queue.
- A value can be null or duplicate.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.queue-1?view=net-7.0

- **Example**

```
using System;
using System.Collections.Generic;

class Order
{
    public int OrderId { get; set; }
    public string CustomerName { get; set; }
    public decimal TotalAmount { get; set; }

    public Order(int orderId, string customerName,
decimal totalAmount)
    {
        OrderId = orderId;
        CustomerName = customerName;
        TotalAmount = totalAmount;
    }

}
```

```
class Program
{
    static void Main()
    {
        Queue<Order> orderQueue = new
Queue<Order>();

        // Adding orders to the queue
        orderQueue.Enqueue(new Order(1, "Alice",
99.99));
        orderQueue.Enqueue(new Order(2, "Bob", 49.99));
        orderQueue.Enqueue(new Order(3, "Charlie",
149.99));


        // Iterating over the queue
        foreach (Order order in orderQueue)
        {
            Console.WriteLine(order);
        }
    }
}
```

- **Stack<T> (System.Collections.Generics)**

In C#, the Stack<T> class is a generic collection class that represents a last-in, first-out (LIFO) collection of objects.

- It provides methods for adding elements to the top of the stack and removing elements from the top of the stack.
- https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.stack-1?view=net-7.0

- **Example**

```
using System;
using System.Collections.Generic;

class Book
{
    public int BookId { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }

    public Book(int bookId, string title, string author)
    {
        BookId = bookId;
        Title = title;
        Author = author;
    }

    public override string ToString()
    {
        return $"Book ID: {BookId}, Title: {Title},
Author: {Author}";
    }
}
```

```
class Program
{
    static void Main()
    {
        Stack<Book> bookStack = new Stack<Book>();

        // Adding books to the stack
        bookStack.Push(new Book(1, "The Great Gatsby",
"F. Scott Fitzgerald"));
        bookStack.Push(new Book(2, "To Kill a
Mockingbird", "Harper Lee"));
        bookStack.Push(new Book(3, "1984", "George
Orwell"));

        // Iterating over the stack
        foreach (Book book in bookStack)
        {
            Console.WriteLine(book);
        }
    }
}
```

References

https://www.c-sharpcorner.com/UploadFile/18585c/overview-of-array-and-collection

https://www.c-sharpcorner.com/UploadFile/skumaar_mca/collections-in-C-Sharp-arraylist-and-arrays/

https://exceptionnotfound.net/csharp-in-simple-terms-13-arrays-and-collections/

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/collections

https://www.tutorialspoint.com/csharp/csharp_arrays.htm

https://www.geeksforgeeks.org/c-sharp-arrays/

https://www.tutorialsteacher.com/csharp/csharp-arraylist